

Software Implementation and Testing Document

For

Group 16

Version 1.0

Authors:

Katya Serechenko
Darya Pylpenko
Logan Bramblett
Narandan Miller

1. Programming Languages (5 points)

Python 3.9+ – used for the entire backend implementation, including:

- REST API implementation (FastAPI routes for backtest, ML, optimizer, auth, stocks, indicators, news, watchlist)
- Three new trading strategy implementations: MACD Crossover, Bollinger Band Breakout, RSI Threshold
- Backtest optimizer with grid search across all five strategies (capped at 100 combinations, ranked by Sharpe ratio)
- Enhanced backtest metrics: CAGR, Sortino ratio, and Calmar ratio (in addition to existing Sharpe ratio and max drawdown)
- ML price direction predictor using scikit-learn (Logistic Regression and Random Forest classifiers) with 7 causal features
- Authentication, database models, and ORM configuration (SQLAlchemy)
- Market data ingestion via Yahoo Finance API (migrated from Stooq)
- Unit tests with pytest covering all new components (42 tests across 6 new test files)

Reason: Python was retained as the sole backend language due to its mature ecosystem for quantitative finance (NumPy, Pandas) and machine learning (scikit-learn). Python 3.9+ compatibility was explicitly verified and enforced throughout Increment 3.

JavaScript (React 18 + Vite) – used for the entire frontend single-page application:

- Dashboard, Ingest, Chart, Indicators, Backtest, and Watchlist views (all retained from Increment 2)
- The BacktestPage exposes SMA Threshold and SMA Crossover strategies in the UI; MACD, Bollinger, RSI Threshold, and Optimizer endpoints are available via the API

Reason: React with Vite was retained from Increment 2 to build on the existing component library and state management patterns.

2. Platforms, APIs, Databases, and other technologies used (5 points)

FastAPI – Used as the backend web framework.

Purpose:

- Defines REST API endpoints (auth, stocks, indicators, backtest, ML, watchlist, news)
- New in Increment 3: POST /backtest/optimize for grid-search optimizer; POST /ml/train/{symbol} and GET /ml/predict/{symbol} for ML

FastAPI was retained for its performance, async support, and automatic OpenAPI documentation.

SQLAlchemy + SQLite – Used as ORM and development database.

Purpose:

- Stores user accounts, stock symbols, and candle price data
- Accessed by BacktestService, OptimizerService, and MLService to retrieve ingested candle data for computation

SQLite was retained for Increment 3 due to its simplicity for local development.

Yahoo Finance API – New in Increment 3; replaces the Stooq market data provider.

Purpose:

- Retrieves historical OHLCV stock price data
- Feeds data into the backtesting engine, indicator calculations, and ML feature builder

Yahoo Finance was selected to replace Stooq due to broader symbol coverage and improved reliability for US equities.

scikit-learn + joblib – New in Increment 3; used for the ML price direction predictor.

Purpose:

- Trains Logistic Regression and Random Forest classifiers on 7 causal features (1-day, 2-day, 3-day, 5-day returns; RSI normalized; MACD histogram; Bollinger Band z-score)
- Evaluates model accuracy, precision, and recall on a held-out temporal test set
- Persists trained models to disk via joblib for subsequent prediction calls

scikit-learn was selected as the industry-standard Python ML library with well-tested implementations of both classifiers. joblib provides efficient model serialization.

NumPy and Pandas – Retained from prior increments.

Purpose:

- Compute technical indicators (SMA, EMA, RSI, MACD, Bollinger Bands) used in strategies and ML features
- Build and manipulate feature matrices for ML training and prediction
- Calculate enhanced backtest metrics (CAGR, Sortino ratio, Calmar ratio)

JWT (JSON Web Tokens) – Retained from prior increments for stateless authentication.

GitHub – Retained for version control and collaboration.

Purpose:

- Hosts source code repository under the Increment3 branch
- Tracks feature development through Issues and pull requests
- CI via GitHub Actions runs pytest on each push

3. Execution-based Functional Testing (10 points)

Functional testing in Increment 3 was performed using **pytest** unit tests targeting all new backend components. Tests were run locally via *pytest backend/tests/* and in CI via GitHub Actions. All tests passed.

test_bollinger_breakout_strategy.py (8 tests)

Tests the `generate_bollinger_breakout_signals()` function for correct BUY/SELL signal generation:

- BUY when close touches or crosses below lower Bollinger Band
- SELL only after a prior BUY when close touches or crosses above upper band
- No signal when price remains between bands
- No duplicate BUY signals while already long
- Correct handling of None (warm-up) band values
- Signal reasons contain the expected human-readable text ("lower band", "upper band")

test_macd_crossover_strategy.py (8 tests)

Tests the `generate_macd_crossover_signals()` function:

- BUY on bullish MACD/signal line crossover (MACD crosses above signal line)
- SELL on bearish crossover only after a prior BUY
- No signals when MACD remains consistently above or below signal throughout
- No duplicate BUY signals while already holding a position
- Correct handling of None values during the MACD warm-up period

test_rsi_threshold_strategy.py (13 tests)

Tests the `generate_rsi_threshold_signals()` function:

- BUY when RSI drops below configurable oversold threshold (default 30)
- SELL when RSI rises above configurable overbought threshold (default 70), only after a prior BUY
- No signal when RSI remains between thresholds
- No signals when all RSI values are None
- No duplicate BUY signals while already long

test_metrics.py (35 tests)

Tests the new enhanced backtest metric calculations:

- Total return calculation on flat, gain, and loss equity curves
- Maximum drawdown detection on monotonic and mixed equity curves
- Sharpe ratio calculation with known risk-free rate assumptions
- Sortino ratio: uses only downside deviation; returns 0 when no downside volatility
- Calmar ratio: annualized return divided by maximum drawdown
- CAGR (Compound Annual Growth Rate) on multi-year equity curves
- Win rate calculation from trade PnL records

test_ml_predictor.py (8 tests)

Tests the `FeatureBuilder` class and ML pipeline:

- Feature matrix shape: X is (n_valid_bars, 7) and y has the same length
- All 7 expected feature names are present in `FEATURE_NAMES`
- No look-ahead bias: features are built strictly from past candles (causal construction)
- Logistic Regression trains and predicts on synthetic candle data without error
- Random Forest trains and predicts on synthetic candle data without error
- Prediction result contains direction, probability, and feature importance fields

test_optimizer.py (7 tests)

Tests the `OptimizerService.run_grid_search()` function:

- Results are sorted in descending order by Sharpe ratio
- `ValueError` is raised when the number of combinations exceeds `MAX_COMBINATIONS` (100)
- `ValueError` is raised for unknown strategy names
- `ValueError` is raised for parameter keys not valid for the selected strategy
- `ValueError` is raised when `param_grid` produces zero combinations
- Single-combination grid search returns exactly one result
- Results contain all required fields (strategy, params, Sharpe ratio, total return, max drawdown, num_trades)

4. Execution-based Non-Functional Testing (10 points)

Non-functional testing in Increment 3 focused on compatibility and robustness.

Python 3.9+ Compatibility Testing

All new backend code was verified to run on Python 3.9 (the minimum supported version for the project). Specific fixes applied:

- Replaced Python 3.10+ union type syntax (`X | Y`) with `typing.Union[X, Y]` and `typing.Optional[X]` throughout new modules
- Replaced dict union syntax (`d1 | d2`) with `{**d1, **d2}` for Python 3.9 compatibility
- Added `from __future__ import annotations` to all new modules requiring PEP 604 style annotations
- The `test_optimizer.py` file explicitly stubs out SQLAlchemy DB models at import time to allow isolated unit testing without a running database

Pydantic v2 Compatibility Testing

Pydantic v2 deprecation warnings were resolved across the codebase:

- Updated schema validators to use Pydantic v2 `model_validator` and `field_validator` decorators
- Replaced deprecated `.dict()` calls with `.model_dump()` in route handlers
- Verified that all request and response schemas serialize correctly under Pydantic v2

Performance / Scalability Guardrails

- The optimizer enforces a hard cap of `MAX_COMBINATIONS` = 100 parameter combinations per grid-search request to prevent runaway computation and ensure API response times remain acceptable
- The ML predictor enforces a minimum of 50 samples before training to ensure statistical validity
- Model files are persisted to disk with `joblib` so repeated prediction calls do not retrain

5. Non-Execution-based Testing (10 points)

Non-execution-based testing in Increment 3 consisted of code reviews conducted via GitHub pull requests.

GitHub Pull Request Code Reviews

All new features were developed on the `Increment3` branch and reviewed by at least one other team member before merging. Reviews focused on:

- Correctness of signal logic in each new strategy (edge cases such as missing position state, `None` indicator values during warm-up periods)
- Correctness of the feature engineering pipeline in `FeatureBuilder` – verifying that no future data leaks into past feature windows
- Correctness of the grid-search optimizer – verifying Sharpe-ratio sorting and `MAX_COMBINATIONS` enforcement
- Correctness of the new metric implementations (Sortino, Calmar, CAGR) against manual calculations
- API contract review: ensuring new endpoints match the OpenAPI schemas defined in `FastAPI`
- Python 3.9 compatibility: reviewers flagged and resolved union type and dict merge syntax issues

Test File Walkthroughs

The team conducted informal walkthroughs of the `pytest` test files to confirm:

- Each test case targets a single, clearly named behavior
- Tests are isolated (no shared mutable state between tests)
- Tests use deterministic synthetic data (fixed seeds, known price sequences) to guarantee repeatable results
- All test assertions are precise (exact signal counts, exact dates, exact signal values)

